

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	K V CHARAN TEJA	Reg. No.:	192424167
Date:		Duration:	60 minutes

Code of Conduct

I K V CHARAN TEJA(192424167) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. Problem Analysis

- Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.

2. Project Structure Design

- Design and explain the folder structure of your project repository.
- Justify the purpose of each major folder and file included in the repository.

3. Git Repository Initialization

- Create a Git repository for the project.
- Explain the steps involved in repository initialization and describe the purpose of the essential Git files.

4. Git Branching Strategy

- Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
- Justify why the selected branching model is appropriate for the assigned project.

5. Version Control Workflow

- Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
- Explain the purpose of each Git operation performed.

6. **Commit History Analysis**
 - Present the commit history of the project.
 - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
 - Explain why Docker is suitable for the assigned project.
 - Identify the application components that require containerization.
8. **Dockerfile Development**
 - Design and explain the Dockerfile required to containerize the application.
 - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
 - Develop a Docker Compose configuration for the project.
 - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
 - Demonstrate the execution of the application using Docker containers.
 - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
 - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
 - Identify the challenges encountered during implementation.
 - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

GIT & DOCKER TECHNICAL ASSIGNMENT

Project: QueueLess – Smart Digital Queue Management System

The following matter is prepared according to the **questions and Excellent-level rubric** given in your Word document. The assignment requires problem analysis, repository structure, Git workflow, Docker implementation, deployment/testing, diagrams, screenshots, challenges, and future enhancements.

1. PROBLEM ANALYSIS

1.1 Problem Statement

Traditional queue management systems require customers to physically stand in long queues and wait for their turn. This results in increased waiting time, overcrowding, inefficient service management, and poor customer experience. Staff members also face difficulties in monitoring queue status and managing customer flow.

QueueLess – Smart Digital Queue Management System is a full-stack web application designed to digitize the queue management process. The system allows customers to join a queue remotely, receive a digital queue number, monitor their position in real time, and receive notifications when their turn is approaching.

Administrators and service providers can create and manage queues, call the next customer, monitor waiting customers, and view queue statistics through an administrative dashboard.

1.2 Objectives

The major objectives of QueueLess are:

1. To eliminate unnecessary physical waiting in queues.
2. To provide customers with digital queue tokens.
3. To display real-time queue position and estimated waiting time.
4. To allow administrators to manage queues efficiently.
5. To provide notifications when the customer's turn is approaching.
6. To maintain queue and transaction information securely.
7. To improve service efficiency and customer satisfaction.
8. To provide a scalable and containerized application using Docker.

1.3 Functional Requirements

Customer Module

- Customer registration and login.
- View available services.
- Select a service and join a queue.
- Generate a digital queue token.
- View current queue position.
- View estimated waiting time.
- Leave/cancel a queue.
- Receive turn notifications.
- View previous queue history.

Admin Module

- Admin authentication.
- Create and manage services.
- Create and manage queues.

- View active customers.
- Call the next customer.
- Skip or remove customers.
- Monitor queue status.
- View service statistics.

System Requirements

- User authentication.
- Real-time queue updates.
- Database storage.
- REST API communication.
- Notification mechanism.
- Error handling and logging.
- Containerized deployment.

1.4 Non-Functional Requirements

- **Performance:** Queue updates should be delivered quickly.
- **Scalability:** The application should support increasing numbers of users and queues.
- **Security:** Authentication and authorization must protect customer and administrator data.
- **Availability:** The system should remain available during normal operating conditions.
- **Maintainability:** The source code should be modular and easy to update.
- **Portability:** The application should run consistently across different environments.
- **Reliability:** Queue numbers and customer positions should be maintained accurately.

1.5 Expected Outcomes

The expected outcome is a working digital queue management platform that reduces physical waiting time and provides real-time queue information. The application should be version-controlled using Git and deployed using Docker containers.

2. PROJECT STRUCTURE DESIGN

A modular repository structure is used to separate the frontend, backend, database configuration, Docker configuration, documentation, and testing components.

QueueLess/

```
|
|
|— frontend/
|
| |— public/
| |
| |— src/
```

```
| | | └── components/
| | | └── pages/
| | | └── services/
| | | └── hooks/
| | | └── assets/
| | └── package.json
| | └── Dockerfile
|
└── backend/
    | └── src/
    | | └── controllers/
    | | └── routes/
    | | └── models/
    | | └── middleware/
    | | └── services/
    | | └── config/
    | └── package.json
    | └── Dockerfile
    |
    └── database/
        | └── migrations/
        | └── seed/
        |
        └── tests/
            | └── frontend/
```

```
|   └── backend/
|
|
|── docs/
|   └── architecture/
|   └── screenshots/
|
|
|── .gitignore
|── .env.example
|── docker-compose.yml
|── README.md
└── LICENSE
```

Purpose of Major Folders

Folder/File	Purpose
frontend/	Contains the user interface of QueueLess
backend/	Contains APIs and business logic
database/	Contains database migration and seed files
tests/	Contains application test cases
docs/	Contains diagrams and documentation
.gitignore	Prevents unnecessary/sensitive files from entering Git
.env.example	Provides required environment-variable format
docker-compose.yml	Defines and manages application containers
README.md	Provides project setup and usage instructions
Dockerfile	Defines how application containers are built

This structure improves **modularity, maintainability, collaboration, scalability, and deployment management**.

3. GIT REPOSITORY INITIALIZATION

Git is used as the version control system for QueueLess. It maintains the complete history of source-code changes and allows developers to work collaboratively.

3.1 Repository Initialization

First, the project directory is created:

```
mkdir QueueLess
```

```
cd QueueLess
```

Initialize Git:

```
git init
```

Configure the Git user:

```
git config user.name "Your Name"
```

```
git config user.email "your@email.com"
```

Check repository status:

```
git status
```

Add project files:

```
git add .
```

Create the first commit:

```
git commit -m "chore: initialize QueueLess project"
```

Connect the local repository with GitHub:

```
git remote add origin https://github.com/username/QueueLess.git
```

Push the project:

```
git branch -M main
```

```
git push -u origin main
```

3.2 Essential Git Files

.gitignore

The .gitignore file prevents unnecessary files from being tracked.

Example:

```
node_modules/
```

.env

dist/

build/

*.log

coverage/

The .env file is excluded because it may contain sensitive information such as database credentials and API keys.

.git Directory

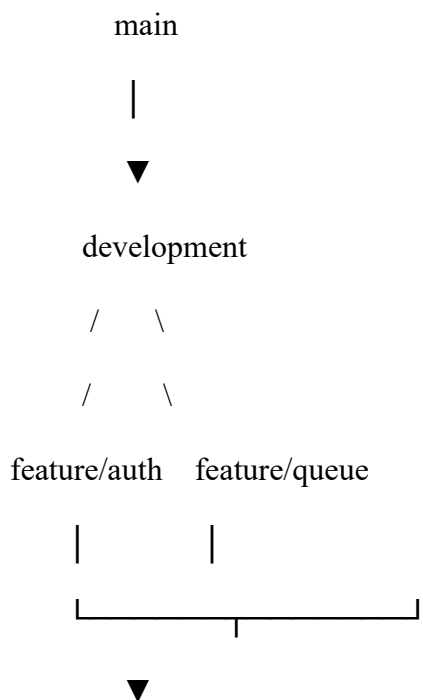
The .git directory stores Git's internal repository information, including commit history, branches, and configuration.

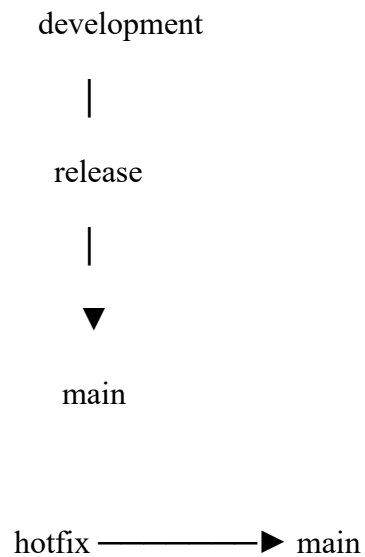
README.md

The README provides information about the project, installation procedure, technologies, commands, and usage.

4. GIT BRANCHING STRATEGY

A structured branching strategy is used to maintain stable and organized development.





Branches Used

Main Branch

The main branch contains stable and production-ready code.

Development Branch

The development branch contains integrated development work before release.

Feature Branch

Feature branches are created for individual functionalities.

Examples:

feature/user-authentication

feature/queue-management

feature/admin-dashboard

feature/notifications

Release Branch

A release branch is used to prepare and test a version before production deployment.

Hotfix Branch

A hotfix branch is created to solve critical production issues.

Git Branch Commands

`git checkout -b development`

Create a feature branch:

`git checkout -b feature/queue-management`

After completing the feature:

`git add .`

`git commit -m "feat: implement queue management"`

Merge the feature:

`git checkout development`

`git merge feature/queue-management`

The branching model is appropriate because it separates stable code from ongoing development and individual features. It reduces the possibility of unstable code being directly introduced into the production branch.

5. VERSION CONTROL WORKFLOW

The QueueLess development workflow follows the following sequence:

Create Feature

↓

Create Feature Branch

↓

Develop & Test

↓

`git add`

↓

`git commit`

↓

Push Feature Branch



Pull Request



Code Review



Merge into Development



Testing



Release



Merge into Main

Important Git Operations

Check Status

`git status`

Shows modified, staged, and untracked files.

Add Changes

`git add .`

Moves changes to the staging area.

Commit

`git commit -m "feat: add digital queue token generation"`

Stores a version of the changes in Git history.

Push

`git push origin feature/queue-management`

Uploads changes to the remote repository.

Pull

`git pull origin development`

Downloads and integrates the latest changes.

Merge

`git checkout development`

`git merge feature/queue-management`

Combines feature development with the development branch.

Conflict Resolution

If a merge conflict occurs:

`git status`

Open the conflicting file and manually select the required changes.

Then:

`git add .`

`git commit -m "fix: resolve merge conflict"`

This workflow provides traceability and supports collaborative development.

6. COMMIT HISTORY ANALYSIS

Professional and meaningful commit messages are used instead of vague messages such as update, changes, or final.

Sample Commit History

a81c92d feat: implement user authentication

b92d431 feat: add digital queue token generation

c73e812 feat: implement admin queue dashboard

d64f123 feat: add real-time queue status

e54a912 fix: correct queue position calculation

f45b823 test: add queue management test cases

g36c714 docs: update project setup instructions

h27d605 chore: add Docker configuration

Commit Message Convention

QueueLess follows a structured convention:

feat: new functionality

fix: bug correction

docs: documentation changes

test: testing changes

chore: configuration/maintenance

refactor: code restructuring

For example:

```
git commit -m "feat: implement digital queue token generation"
```

is better than:

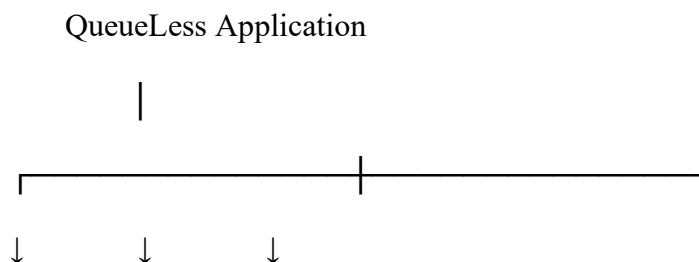
```
git commit -m "changes"
```

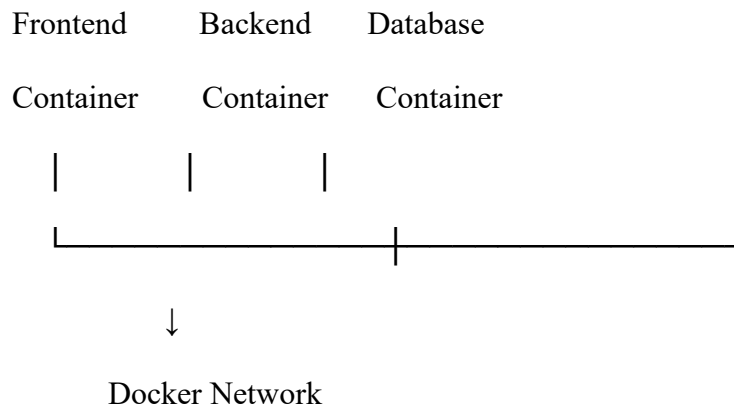
Meaningful commits make the project history easier to understand, debug, maintain, review, and revert.

7. DOCKER ENVIRONMENT DESIGN

Docker is suitable for QueueLess because the application contains multiple components that need to operate together.

The major components are:





Components Requiring Containerization

Frontend Container

Hosts the QueueLess user interface.

Backend Container

Runs REST APIs, authentication, queue management, and business logic.

Database Container

Stores users, services, queue tokens, queue status, and other application information.

Benefits of Docker

Docker provides:

- Environment consistency.
- Portability.
- Easy deployment.
- Dependency isolation.
- Scalability.
- Faster application setup.
- Easier testing.
- Reproducible environments.

Instead of manually installing all dependencies, developers can start the complete application using Docker Compose.

8. DOCKERFILE DEVELOPMENT

A Dockerfile contains the instructions required to create an application image.

Backend Dockerfile

FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 5000

CMD ["npm", "start"]

Explanation

FROM

FROM node:20-alpine

Provides Node.js as the base environment.

WORKDIR

WORKDIR /app

Sets the working directory inside the container.

COPY

COPY package*.json ./

Copies package configuration files.

RUN

RUN npm install

Installs application dependencies.

COPY

COPY . .

Copies the remaining application source code.

EXPOSE

EXPOSE 5000

Documents the port used by the backend.

CMD

CMD ["npm", "start"]

Starts the application when the container runs.

Building the Image

docker build -t queueless-backend .

Run the container:

docker run -p 5000:5000 queueless-backend

9. DOCKER COMPOSE DESIGN

Docker Compose is used to define and run multiple containers together.

docker-compose.yml

services:

frontend:

build: ./frontend

ports:

- "3000:3000"

depends_on:

- backend

networks:

- queueless-network

backend:

build: ./backend

ports:

- "5000:5000"

environment:

DATABASE_URL: mongodb://database:27017/queueless

depends_on:

- database

networks:

- queueless-network

database:

image: mongo:7

ports:

- "27017:27017"

volumes:

- database-data:/data/db

networks:

- queueless-network

networks:

queueless-network:

volumes:

database-data:

Services

Frontend

Runs the QueueLess user interface on port 3000.

Backend

Runs the application API on port 5000.

Database

Provides persistent database storage.

Networking

All services are connected using:

queueless-network

Therefore, containers can communicate with each other using their service names.

For example:

backend → database

can use the database service name instead of a hard-coded IP address.

Volumes

volumes:

database-data:

The volume preserves database information even when the database container is recreated.

Dependencies

depends_on:

ensures that dependent services are started in the required order.

Starting the Application

`docker compose up --build`

Run in background:

`docker compose up -d --build`

Check running containers:

`docker compose ps`

Stop the application:

`docker compose down`

10. CONTAINER DEPLOYMENT AND TESTING

10.1 Build and Start

The complete application is started using:

`docker compose up --build`

Expected output should show the frontend, backend, and database containers being created and started.

10.2 Verify Containers

`docker ps`

Expected containers:

queueless-frontend

queueless-backend

queueless-database

10.3 Test Frontend

Open the application in a browser:

`http://localhost:3000`

The QueueLess login/home page should be displayed.

10.4 Test Backend

The backend can be tested through its API endpoint:

http://localhost:5000

or through the appropriate health-check API.

10.5 Test Database

The backend should successfully connect to the database container.

10.6 Functional Testing

The following operations should be tested:

Test Case	Expected Result
User registration	User account created
User login	User successfully authenticated
Select service	Available service displayed
Join queue	Digital token generated
View queue	Current position displayed
Call next customer	Queue moves to next customer
Leave queue	Customer removed from active queue
Admin dashboard	Queue information displayed

Evidence/Screenshots to Include

For maximum rubric coverage, include screenshots of:

1. git init
2. git status
3. Git commit history
4. Git branches
5. GitHub repository
6. Feature branch
7. Merge operation
8. Docker image build
9. docker compose up
10. docker ps
11. QueueLess application running
12. Admin dashboard
13. Customer queue/token page

The rubric specifically expects screenshots and evidence demonstrating successful deployment and testing.

11. BENEFITS OF GIT AND DOCKER

11.1 Collaboration

Git allows multiple developers to work on different features simultaneously using branches.

11.2 Version Management

Git maintains a complete history of project modifications. Developers can identify who made a change and when it was made.

11.3 Portability

Docker packages the application and its dependencies into containers, allowing the application to run consistently across different systems.

11.4 Deployment

Docker simplifies deployment because the same container configuration can be used during development, testing, and deployment.

11.5 Scalability

Individual services can be scaled according to demand. For example, additional backend containers can be deployed when the number of users increases.

11.6 Maintainability

Git provides structured version history while Docker separates application components and dependencies.

11.7 Reliability

Containerized environments reduce differences between development and production environments.

11.8 Faster Development

Developers can clone the repository and run:

```
docker compose up --build
```

to start the required services without manually configuring every dependency.

Git and Docker therefore support the major objectives of the assignment, particularly **collaboration, version management, portability, deployment, scalability, and maintainability**.

12. CHALLENGES AND IMPROVEMENTS

12.1 Challenges Encountered

1. Git Merge Conflicts

Different developers may modify the same file, resulting in merge conflicts.

Solution:

Use feature branches, regular pulls, and proper conflict-resolution procedures.

2. Docker Dependency Issues

Incorrect package versions or missing dependencies may prevent containers from starting.

Solution:

Use appropriate Docker base images and maintain dependency files correctly.

3. Database Connectivity

The backend may initially fail to connect to the database container.

Solution:

Use the Docker Compose service name as the database hostname and configure environment variables correctly.

4. Port Conflicts

A port may already be occupied by another application.

Solution:

`docker ps`

Identify the conflicting service and change the host port if required.

5. Environment Variables

Sensitive configuration information should not be directly committed to Git.

Solution:

Use `.env` files locally and provide `.env.example` without sensitive values.

FUTURE ENHANCEMENTS

The QueueLess system can be improved in several ways:

1. **AI-based waiting-time prediction**

Machine learning can estimate waiting time based on historical queue patterns.

2. **Mobile Application**

A dedicated Android/iOS application can provide easier access to queue services.

3. **Push Notifications**

Customers can receive notifications when their turn is approaching.

4. **Cloud Deployment**

The Dockerized application can be deployed on cloud platforms.

5. **Auto Scaling**

Backend services can automatically scale based on user traffic.

6. **Analytics Dashboard**

Administrators can analyze peak hours, average waiting time, service efficiency, and customer volume.

7. **CI/CD Pipeline**

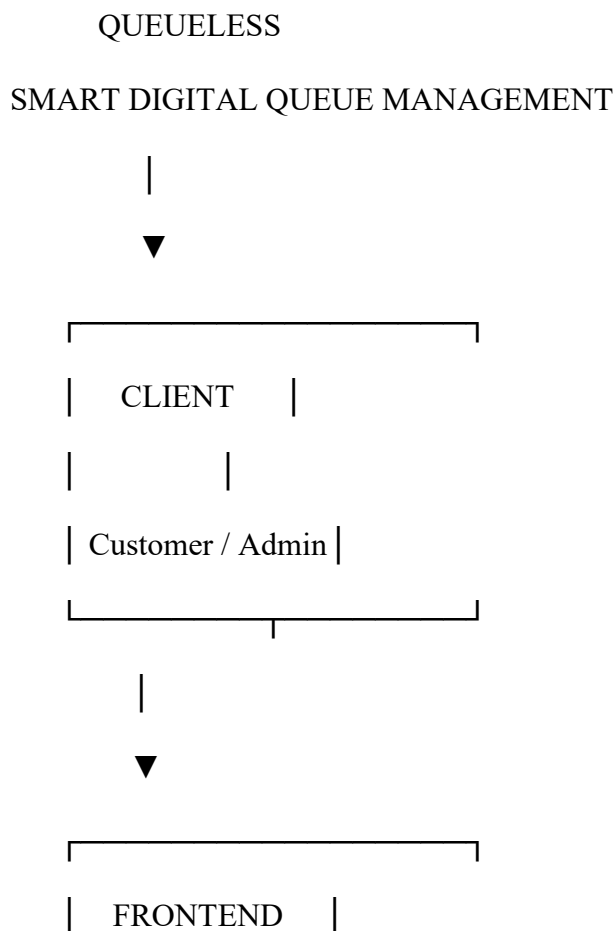
GitHub Actions can automatically build, test, and deploy the application whenever changes are pushed.

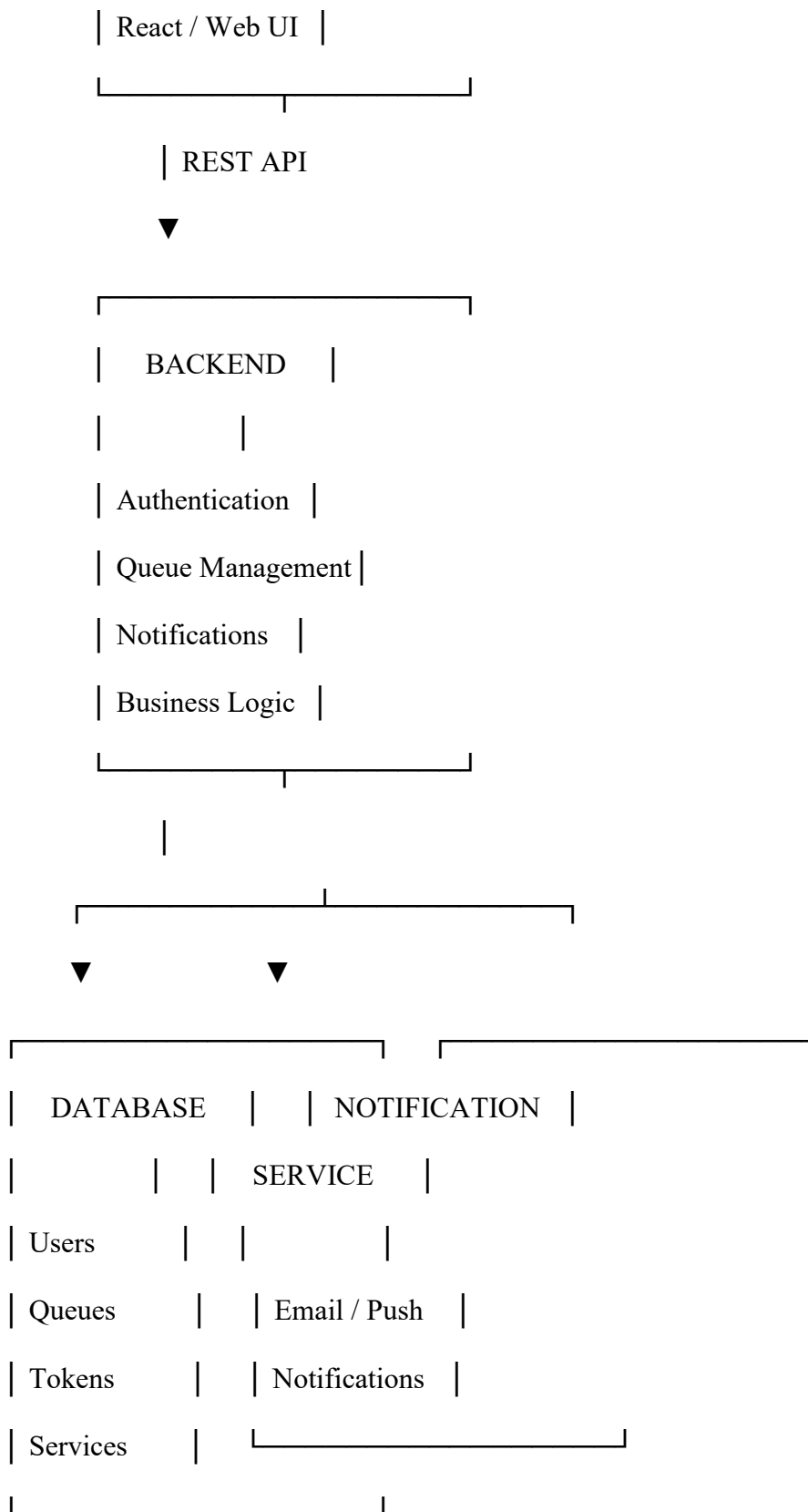
8. **Advanced Security**

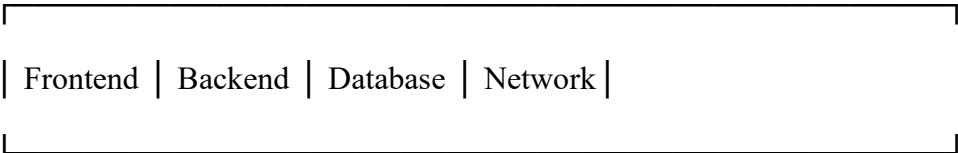
OAuth, JWT improvements, rate limiting, encryption, and role-based access control can strengthen security.

ARCHITECTURE DIAGRAM

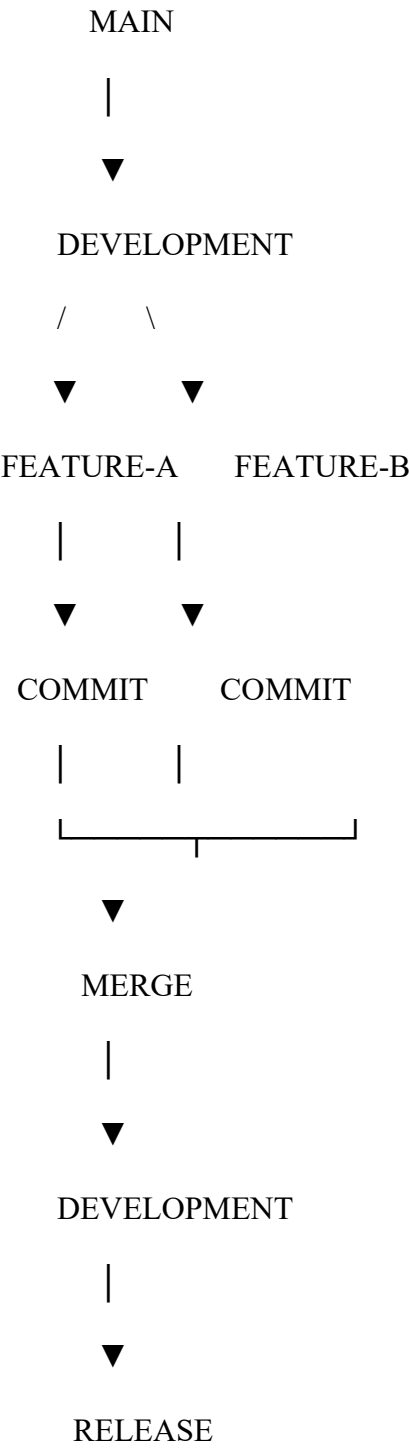
You can use this structure for your architecture diagram:







GIT WORKFLOW DIAGRAM





MAIN

DOCKER DEPLOYMENT WORKFLOW

Source Code



Git Repository



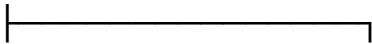
Dockerfile



Docker Image



Docker Compose



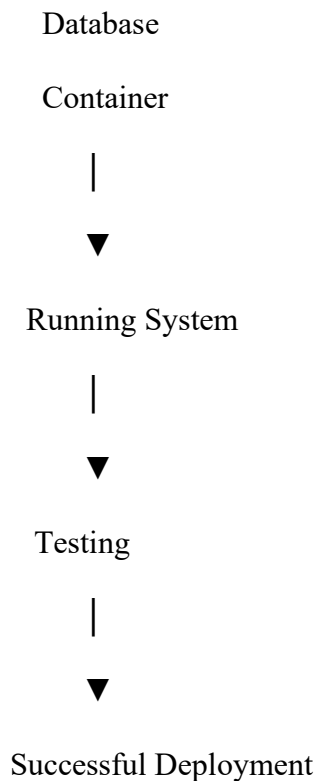
Frontend

Backend

Container

Container





CONCLUSION

QueueLess provides a digital solution for managing service queues efficiently. The proposed system reduces physical waiting, improves customer experience, and provides administrators with better control over queue operations.

Git provides systematic version control, collaboration, branching, commit history, and synchronization. Docker provides environment consistency, portability, dependency isolation, and simplified deployment. Docker Compose further simplifies the management of the frontend, backend, and database services.

Therefore, combining **Git for version control** and **Docker for containerization** provides a reliable and maintainable development and deployment approach for the QueueLess application. This directly addresses the CO3 objective of building full-stack applications with emphasis on **containerization, version control, and collaborative development**.

Rubric coverage

This content is structured to target the **Excellent** level by covering the required areas: detailed problem analysis and project design, proper Git repository and branching strategy, meaningful commits and workflow, Dockerfile and Compose configuration, deployment/testing evidence, and professional documentation with diagrams, screenshots, benefits, challenges, and future improvements.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

